

JOBSHEET WEEK 1 WEB PROGRAMMING



SIMPUS-Mini

HTML5 Page Structure for a Mini Library System

**DEPARTMENT OF INFORMATION TECHNOLOGY
STATE POLYTECHNIC OF MALANG
2026**

Table of Contents

Introduction	4
About This Program	4
Folder Structure.....	4
1. Core Concepts Used in This Jobsheet	5
1.1 What Are a "Tag" and an "Element"?	5
1.2 The Required Structure of Every HTML Page.....	5
1.3 HTML5 Semantic Tags.....	5
1.4 Why Is There No CSS Yet?.....	6
1.5 Navigation Between Pages ()	6
2. Explanation of <code>index.html</code> (Home Page)	7
2.1 Full Code	7
2.2 Section-by-Section Explanation	8
<header> — Page Header.....	8
<nav> — Navigation Menu.....	8
<main> — Main Content	8
<footer> — Page Footer.....	9
2.3 Conclusion	9
3. Explanation of <code>books/list.html</code> (Book List).....	10
3.1 Page Skeleton	10
3.2 Anatomy of an HTML Table	10
The "Actions" Column.....	11
3.3 The Data Displayed (Dummy)	11
3.4 Why Use <table>, Instead of Stacked <div> Elements?	11
4. Explanation of <code>books/add.html</code> (Add Book Form).....	13
4.1 Full Form Code.....	13
4.2 Element: <form>	13
4.3 The Pattern for Every Form Field: <label> + <input>	14
4.4 Types of Input Used	14
<select> and <option> — Dropdown Selection	14
4.5 The Submit Button.....	15
4.6 Summary of the Learning Flow	15
5. Explanation of <code>members/list.html</code> (Member List).....	16
5.1 Full Code	16
5.2 What Is the Same as <code>books/list.html</code> ?	16
5.3 What Is Different?	16
5.4 Take Note: The Navigation Menu	17
6. Explanation of <code>members/add.html</code> (Add Member Form).....	18
6.1 Full Code	18
6.2 What Is the Same as <code>books/add.html</code> ?	18

6.3 What Is Different?	18
6.4 Why Is "Member No." Text and Not a Number?	19
6.5 Reflective Exercise	19
7. Summary & Further Exercises	20
7.1 Overall Summary of Jobsheet 1	20
7.2 Remember: This Is Only the Skeleton.....	20
7.3 How to Try It Yourself	20
7.4 Exercises	21

Introduction

Jobsheet 1 builds the initial skeleton of the **SIMPUS-Mini** application, a mini library system that can manage **book** and **member** data. At this early stage the application cannot store real data yet. All the data in the tables is still *dummy* (sample) data typed manually into the HTML, and the forms are not processed anywhere yet. The focus of this jobsheet is:

- Web page structure using **semantic HTML5** (**header**, **nav**, **main**, **section**, **article**, **footer**), not plain **div** everywhere.
- Naming the **id** and **name** attributes on forms, which will be used again when learning CSS (jobsheet 2) and JavaScript/backend (the following jobsheets).

Folder Structure

```
jobsheet-01/
├── index.html           # Home page
├── books/
│   ├── list.html       # Book list table (static)
│   └── add.html         # Add book form (not processed yet)
├── members/
│   ├── list.html       # Member list table (static)
│   └── add.html         # Add member form (not processed yet)
├── README.md           # Short jobsheet summary (from the lecturer)
└── documentation/      # This documentation folder
```

1. Core Concepts in the Week 1 Jobsheet

Before moving on to the explanation of each file, let us first get to know the terms that will be used repeatedly across all the HTML files in this jobsheet.

1.1 What Are a "Tag" and an "Element"?

HTML is written using **tags** that usually come in pairs: an opening tag `<tagname>` and a closing tag `</tagname>`. Everything from the opening tag to the closing tag is called an **element**. For example:

```
<h1>SIMPUS-Mini</h1>
```

`<h1>` is the opening tag, `</h1>` is the closing tag, and the whole snippet above is an **h1** element (a level-1 heading).

1.2 The Required Structure of Every HTML Page

Every file in this jobsheet begins with the same pattern:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>SIMPUS-Mini | Home</title>
</head>
<body>
  ...
</body>
</html>
```

Part	Function
<code><!DOCTYPE html></code>	Tells the browser that this document is HTML5. It must be on the very first line.
<code><html lang="en"></code>	The element that wraps the whole page. <code>lang="en"</code> tells the browser (and screen readers) that the page content is in English.
<code><head></code>	Contains information <i>about</i> the page that is not displayed on screen: the tab title, character encoding, CSS links, and so on.
<code><meta charset="UTF-8"></code>	Sets the character encoding to UTF-8 so that characters such as <code>é</code> , <code>&mdash;</code> , or non-Latin letters display correctly instead of turning into strange symbols.
<code><title></code>	The text that appears in the browser tab . Notice that each page has a different title (<code>SIMPUS-Mini Home</code> , <code>SIMPUS-Mini Book List</code> , and so on) so that users know which page they are on.
<code><body></code>	Contains all the content that is displayed on screen: text, images, tables, forms, and so on.

1.3 HTML5 Semantic Tags

"Semantic" means the tag name describes the **meaning/role** of its content, rather than being just an empty box like `<div>`. These are the tags used in this jobsheet:

- **<header>** : the head of the page, usually holding the site title and the navigation. It appears on **all** pages of this jobsheet with similar content (the **SIMPUS-Mini** title + the **nav** menu).
- **<nav>** : a group of navigation links (a menu). Inside it we use ordinary **//<a>** elements, but they are wrapped in **<nav>** so that the *browser/screen reader* knows this is a "menu", not just an ordinary list.
- **<main>** : the **main** content of the page. There may be only **one** **<main>** per page.
- **<section>** : groups content that shares one theme, usually starting with a heading (**<h2>**, and so on).
- **<article>** : a self-contained piece of content that could be separated from its surroundings. In **index.html**, each statistics card (Total Books, Total Members, On Loan) is wrapped in **<article>** because each one is a standalone piece of information.
- **<footer>** : the foot of the page, usually holding *copyright*/additional info. It is the same on every page: © 2026 **SIMPUS-Mini** – Jobsheet 1.

Why not just use **<div>** for everything? Because **<div>** carries no meaning at all. Browsers, search engines, and assistive technology (*screen reader*) cannot tell whether a given **<div>** is a menu, the main content, or a footer. With semantic tags the code is easier to read and friendlier to accessibility and SEO.

1.4 Why Is There No CSS Yet?

As noted in this jobsheet's README.md, this stage deliberately has no CSS/JavaScript yet. The goal is to focus on the structure (the skeleton) of the page first. Because there is no CSS, the browser view will look plain (default font, no colour or tidy layout). *Styling* will be covered in the next jobsheet.

1.5 Navigation Between Pages ()

Notice that the **<nav>** menu in each file uses a different relative path depending on where that file is located, for example:

- From **index.html** (in the root folder) to the book list: **href="books/list.html"** (going down into the **books** folder).
- From **books/list.html** back to the home page: **href="../index.html"** (**../** means going up one folder).
- From **books/list.html** to **books/add.html**: just **href="add.html"** because they are both in the same folder.

This is important to understand, because relative path mistakes are one of the most common reasons links "do not work" when learning HTML.

2. Explanation of `index.html` (Home Page)

This file is the **first page** opened when the application runs (because it is named `index.html`, the standard name browsers and servers automatically look for as the main page of a folder).

2.1 Full Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>SIMPUS-Mini | Home</title>
</head>
<body>
  <header>
    <h1>SIMPUS-Mini</h1>
    <nav>
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="books/list.html">Book List</a></li>
        <li><a href="books/add.html">Add Book</a></li>
        <li><a href="members/list.html">Member List</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section>
      <h2>Welcome to the Mini Library System</h2>
      <p>A simple application for managing library book and member data.</p>
    </section>

    <section>
      <h2>Summary</h2>
      <article>
        <h3>Total Books</h3>
        <p>12</p>
      </article>
      <article>
        <h3>Total Members</h3>
        <p>8</p>
      </article>
      <article>
        <h3>On Loan</h3>
        <p>3</p>
      </article>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 SIMPUS-Mini &mdash; Jobsheet 1</p>
  </footer>
</body>
</html>
```

2.2 Section-by-Section Explanation

<header> : Page Header

```
<header>
  <h1>SIMPUS-Mini</h1>
  <nav>...</nav>
</header>
```

- `<h1>SIMPUS-Mini</h1>` is the main title of the application. Every HTML page should have only **one** `<h1>`, which is the highest-level heading.
- Inside `<header>` there is a `<nav>` holding navigation links to 4 pages: Home, Book List, Add Book, and Member List.

`<nav>` : Navigation Menu

```
<nav>
  <ul>
    <li><a href="index.html">Home</a></li>
    <li><a href="books/list.html">Book List</a></li>
    <li><a href="books/add.html">Add Book</a></li>
    <li><a href="members/list.html">Member List</a></li>
  </ul>
</nav>
```

- `<ul>` (unordered list) wraps the menu list.
- Each `<li>` (list item) is one menu item holding one `<a>` link.
- `href` on each `<a>` determines the destination page. Because `index.html` sits in the **root** folder of jobsheet-01, the paths to the `books/` and `members/` folders are written directly, without `../`.

`<main>` : Main Content

It holds two `<section>`:

Section 1: Welcome

```
<section>
  <h2>Welcome to the Mini Library System</h2>
  <p>A simple application for managing library book and member data.</p>
</section>
```

Just a welcome message: one `<h2>` (a sub-section heading, one level below `<h1>`) and one `<p>` paragraph.

Section 2: Statistics Summary

```
<section>
  <h2>Summary</h2>
  <article>
    <h3>Total Books</h3>
    <p>12</p>
  </article>
  <article>
    <h3>Total Members</h3>
    <p>8</p>
  </article>
  <article>
    <h3>On Loan</h3>
    <p>3</p>
  </article>
</section>
```

- Each `<article>` here represents one self-contained statistics "card": a small heading (`<h3>`, one level below `<h2>`) + a number (`<p>`).

- The numbers 12, 8, 3 here are **only sample (dummy) data** typed in manually. There is no logic yet that calculates them from real data; that comes later, with server-side programming and JavaScript.
- Why choose `<article>` and not `<div>`? Because each card can "stand on its own" in meaning: if it were moved to another page, the information would still be complete and make sense (for example "Total Books: 12").

`<footer>` : Page Footer

```
<footer>
  <p>&copy; 2026 SIMPUS-Mini &mdash; Jobsheet 1</p>
</footer>
```

- `©` is an **HTML entity** (a special character code) for the © symbol.
- `—` is the entity for the long dash – (em dash). Entities are used because some symbol characters cannot always be typed directly and safely in HTML, so they are written in this coded form.

2.3 Conclusion

`index.html` shows the basic pattern that is **repeated** on every other page in this jobsheet: `header` (title + nav) → `main` (page-specific content) → `footer` (copyright). Once you understand this file, the other 4 HTML files will be much easier, because the skeleton is identical and only the contents of `<main>` differ.

3. Explanation of `books/list.html` (Book List)

This file displays a **table** listing the library books (static sample data, 5 rows).

3.1 Page Skeleton

The `<header>` and `<footer>` sections follow **exactly the same** pattern as `index.html` (see the `index.html` documentation); only the paths in `href` change, because this file sits inside the `books/` folder rather than the root:

```
<li><a href="../index.html">Home</a></li>
<li><a href="list.html">Book List</a></li>
<li><a href="add.html">Add Book</a></li>
<li><a href="../members/list.html">Member List</a></li>
```

- `../index.html` → go up one folder to the root, then open `index.html`.
- `list.html` and `add.html` → no prefix, because both sit in the same `books/` folder.
- `../members/list.html` → go up one folder to the root, then down into the `members/` folder.

The genuinely new part of this file is inside `<main>`, namely the **HTML table**.

3.2 Anatomy of an HTML Table

```
<table>
  <thead>
    <tr>
      <th>Title</th>
      <th>Author</th>
      <th>Year</th>
      <th>Stock</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Laskar Pelangi</td>
      <td>Andrea Hirata</td>
      <td>2005</td>
      <td>4</td>
      <td>
        <button type="button">Edit</button>
        <button type="button">Delete</button>
      </td>
    </tr>
    <!-- ... 4 more rows with the same pattern ... -->
  </tbody>
</table>
```

Tag	Stands For	Function
<code><table></code>	table	Wrapper for the whole table.
<code><thead></code>	table head	The table head section, holding the column names.
<code><tbody></code>	table body	The table body section, holding the actual data rows.
<code><tr></code>	table row	One row of the table (horizontal).
<code><th></code>	table header cell	One column heading cell (text is automatically bold and centred by the browser).
<code><td></code>	table data cell	One ordinary data cell inside a row.

How to read the structure: `<table>` holds rows (`<tr>`), and each row holds cells (`<th>` for headers, `<td>` for data). So this table has 5 columns (Title, Author, Year, Stock, Actions) and 5 rows of book data.

The "Actions" Column

```
<td>
  <button type="button">Edit</button>
  <button type="button">Delete</button>
</td>
```

- Every row has 2 buttons: **Edit** and **Delete**.
- `type="button"` means this is an **ordinary button**, not a form submit button (unlike the "Save" button on the form pages). If `type` is not written and this button sits **inside** a `<form>`, the browser treats it as a submit button by default.
- **Important to understand:** these buttons **do not do anything yet**. There is no JavaScript connecting a button click to an edit or delete action. This is only the display skeleton (UI); the logic follows in the next jobsheet, when we learn JavaScript.

3.3 The Data Displayed (Dummy)

There are 5 rows of book data typed manually straight into the HTML:

Title	Author	Year	Stock
Laskar Pelangi	Andrea Hirata	2005	4
Bumi Manusia	Pramoedya Ananta Toer	1980	2
Negeri 5 Menara	Ahmad Fuadi	2009	0
Filosofi Teras	Henry Manampiring	2018	5
Ronggeng Dukuh Paruk	Ahmad Tohari	1982	1

Because this is **static HTML** (not connected to a database yet), this data cannot be added to or changed through the application. To add a new row you have to edit the HTML code directly (copy and paste one `<tr>...</tr>` block, then replace its contents).

3.4 Why Use `<table>`, Instead of Stacked `<div>` Elements?

A table is the right tag choice here because the data really is **tabular**, with clearly related columns and rows (each row is one book, each column is one book attribute). Browsers and screen readers also understand the `<table>` structure specifically (they can announce something like "Year column, row 2, contents 1980"), which you do not get with plain `<div>` elements.

4. Explanation of `books/add.html` (Add Book Form)

This file displays a **form** for adding new book data. It is the first file in this jobsheet to introduce the `<form>` element and the various types of `<input>`.

4.1 Full Form Code

```
<form>
  <p>
    <label for="title">Title</label><br>
    <input type="text" id="title" name="title" required>
  </p>
  <p>
    <label for="author">Author</label><br>
    <input type="text" id="author" name="author" required>
  </p>
  <p>
    <label for="year">Year Published</label><br>
    <input type="number" id="year" name="year" min="1900" max="2026" required>
  </p>
  <p>
    <label for="isbn">ISBN</label><br>
    <input type="text" id="isbn" name="isbn">
  </p>
  <p>
    <label for="stock">Stock</label><br>
    <input type="number" id="stock" name="stock" min="0" required>
  </p>
  <p>
    <label for="category">Category</label><br>
    <select id="category" name="category">
      <option value="fiction">Fiction</option>
      <option value="non-fiction">Non-Fiction</option>
      <option value="reference">Reference</option>
    </select>
  </p>
  <p>
    <button type="submit">Save</button>
  </p>
</form>
```

4.2 Element: `<form>`

```
<form>
  ...
</form>
```

`<form>` wraps all the input elements that **will be sent together** when the submit button is pressed. Notice that the `<form>` tag here **has no `action` or `method` attribute**, which means that if the "Save" button is pressed now, the form **does not send data anywhere yet** (the browser simply reloads the same page). This matches the README note: the form is "not processed yet". Connecting the form to data storage (`action="..."`, `method="post"`, and then a backend that processes it) is material for the next jobsheet.

4.3 The Pattern for Every Form Field: `<label>` + `<input>`

Each field is wrapped in a `<p>` (paragraph) and holds a `<label>`—`<input>` pair, for example:

```
<p>
  <label for="title">Title</label><br>
```

```
<input type="text" id="title" name="title" required>
</p>
```

- **<label for="title">Title</label>** : the caption text "Title" shown beside or above the input box. The **for="title"** attribute **connects** this label to the input whose **id** is **title**. The benefit: when the user clicks the label text "Title", the browser automatically puts the cursor in its input box. This is an important practice for accessibility and user comfort (especially on phones, where it makes the tap area larger).
- **
** : a line break, moving the input onto a new line below the label instead of sitting to its right.
- **<input>** : the input box itself. Its important attributes:
 - **type="text"** : an input for free text.
 - **id="title"** : the unique identity of this element on the page, used to link it to the **<label for="title">** above.
 - **name="title"** : the name of the field that will be sent to the server when the form is submitted (for example: **title=Laskar+Pelangi**). **id** and **name** are deliberately given the **same value** in this form for convenience, but they serve different purposes: **id** links to the **<label>/CSS/JS on this page**, while **name** is the **data that gets sent** to the server.
 - **Required** : a built-in HTML5 validation attribute. The browser refuses to submit the form (and shows a warning message) if this field is left empty.

4.4 Types of Input Used

Field	Code	Explanation
Title	<code><input type="text" ... required></code>	Free text, required.
Author	<code><input type="text" ... required></code>	Free text, required.
Year Published	<code><input type="number" min="1900" max="2026" required></code>	Accepts numbers only. min/max restrict the range to sensible years (1900–2026) directly in the browser, with no extra JavaScript.
ISBN	<code><input type="text" ... ></code> (without required)	Free text, may be left empty , because not all older books have an ISBN.
Stock	<code><input type="number" min="0" required></code>	A number, minimum 0 (stock cannot be negative), required.
Category	<code><select></code> with <code><option></code>	See the explanation below.

<select> and <option> — Dropdown Selection

```
<select id="category" name="category">
  <option value="fiction">Fiction</option>
  <option value="non-fiction">Non-Fiction</option>
  <option value="reference">Reference</option>
</select>
```

- **<select>** creates a **dropdown menu** (a selection box that can be clicked to open a list of options).

- Each `<option>` is one choice inside that dropdown.
 - `value="fiction"` : the value that is **actually sent** to the server when it is selected.
 - `Fiction` (the text between the tags) : what is **displayed** to the user on screen.
 - The `value` and the displayed text **do not have to be the same**, but here they happen to be similar (`non-fiction` vs `Non-Fiction`) so that they stay consistent and easy to read, both in the code and on screen.
- Unlike the other fields, the `<select>` here is **not** given `required`, because an option is always selected by default (the first one, `Fiction`), so this field is automatically "filled in".

4.5 The Submit Button

```
<button type="submit">Save</button>
```

- `type="submit"` makes this button, when clicked, **send the entire contents of the form** (following the `action/method` rules on the `<form>` tag : which are currently empty). This differs from the `type="button"` button in `books/list.html`, which does not submit anything.
- Because `<form>` does not have an `action` yet, pressing "Save" right now only reloads the same page, with no visible effect.

4.6 Summary of the Learning Flow

The important sequence to remember from this file:

1. `<form>` = the container for sending data.
2. `<label for="...">` is paired with the matching `<input id="...">`.
3. `name="..."` determines the name of the data sent to the server.
4. HTML5 attributes (`required`, `min`, `max`) provide basic validation **without JavaScript**.
5. `<select>/<option>` are used when the user's choice is limited to a few fixed options rather than free text.

5. Explanation of `members/list.html` (Member List)

This file is an **independent assignment** in Jobsheet 1. Its pattern is deliberately similar to `books/list.html` so that you can apply the HTML table concept yourself to different data (members, not books).

5.1 Full Code

```
<table>
  <thead>
    <tr>
      <th>Member No.</th>
      <th>Name</th>
      <th>Address</th>
      <th>Phone No.</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>A001</td>
      <td>Siti Aminah</td>
      <td>Malang</td>
      <td>0812xxxx</td>
      <td>
        <button type="button">Edit</button>
        <button type="button">Delete</button>
      </td>
    </tr>
    <tr>
      <td>A002</td>
      <td>Budi Santoso</td>
      <td>Batu</td>
      <td>0813xxxx</td>
      <td>
        <button type="button">Edit</button>
        <button type="button">Delete</button>
      </td>
    </tr>
  </tbody>
</table>
```

5.2 What Is the Same as `books/list.html`?

The structure is **identical** to the book list: `<table>` → `<thead>` (the column heading row) → `<tbody>` (the data rows) → each `<tr>` row holds `<td>` cells, and the last column "Actions" holds two `type="button"` buttons (Edit & Delete) that also **do not work yet**.

5.3 What Is Different?

The only differences are the **column names** and **the data**, because the entity being displayed is different (members, not books):

Book Columns	Member Columns
Title	Member No.
Author	Name
Year	Address

Book Columns	Member Columns
Stock	Phone No.
Actions	Actions

The data displayed (2 rows, fewer than the 5 rows in the book list):

Member No.	Name	Address	Phone No.
A001	Siti Aminah	Malang	0812xxxx
A002	Budi Santoso	Batu	0813xxxx

The phone numbers are deliberately written as **0812xxxx/0813xxxx** (not real, complete numbers) because this is only **sample data** for practice, not real personal data — good practice when creating example or dummy data.

5.4 Take Note: The Navigation Menu

```
<nav>
  <ul>
    <li><a href="../index.html">Home</a></li>
    <li><a href="../books/list.html">Book List</a></li>
    <li><a href="list.html">Member List</a></li>
    <li><a href="add.html">Add Member</a></li>
  </ul>
</nav>
```

Notice that this file holds 4 navigation links, including **"Add Member"** pointing to **add.html**, unlike the menus in **index.html** and **books/*.html**, which do not include that link. This is part of the independent assignment: make sure the menu on every page **consistently** shows all 5 pages of this jobsheet (Home, Book List, Add Book, Member List, Add Member). As an exercise, check and complete the menus yourself in **index.html**, **books/list.html**, and **books/add.html** so that both the **"Member List"** and **"Add Member"** links appear on every page.

6. Explanation of `members/add.html` (Add Member Form)

Just like `members/list.html`, this file is also part of the **independent assignment**: its pattern follows `books/add.html`, but for member data.

6.1 Full Code

```
<form>
  <p>
    <label for="name">Name</label><br>
    <input type="text" id="name" name="name" required>
  </p>
  <p>
    <label for="member_no">Member No.</label><br>
    <input type="text" id="member_no" name="member_no" required>
  </p>
  <p>
    <label for="address">Address</label><br>
    <input type="text" id="address" name="address">
  </p>
  <p>
    <label for="phone_no">Phone No.</label><br>
    <input type="text" id="phone_no" name="phone_no">
  </p>
  <p>
    <button type="submit">Save</button>
  </p>
</form>
```

6.2 What Is the Same as `books/add.html`?

The basic pattern is identical to the add book form: every field is wrapped in `<p>`, holds a pair of `<label for="...">` + `<input id="..." name="...">`, and ends with a `<button type="submit">Save</button>` button.

6.3 What Is Different?

Fewer and simpler fields are required than in the book form. **All** the fields here are `type="text"` (there is no `type="number"` or `<select>`, because the member data in this jobsheet needs no numbers and no category choice):

Field	Input Code	Required?
Name	<code><input type="text" id="name" name="name" required></code>	Yes (required)
Member No.	<code><input type="text" id="member_no" name="member_no" required></code>	Yes (required)
Address	<code><input type="text" id="address" name="address"></code>	No
Phone No.	<code><input type="text" id="phone_no" name="phone_no"></code>	No

Notice that the `id/name` attribute naming uses an **underscore** (`member_no`, `phone_no`) because the field name is made of several words. This is a common naming convention in HTML and programming, so that variable names contain no spaces (a space in `name` can cause problems when the data is sent or processed).

6.4 Why Is "Member No." Text and Not a Number?

The "Member No." field uses `type="text"`, not `type="number"`, even though its name holds the word "number". This is because the member number format in this application is a combination of letters and digits, for example `A001`, `A002` (see the data in `members/list.html`), and `type="number"` accepts pure digits only — it cannot hold the letter `A`. This is a concrete example of why you should choose the input `type` according to the **actual shape of the data**, not just its name.

6.5 Reflective Exercise

As independent practice, compare this form with the book form yourself and answer the following questions to test your understanding:

1. Why are the "Address" and "Phone No." fields not given `required`, while "Name" and "Member No." are?
2. What happens (in the browser) if you click the "Save" button without filling in the "Name" field? Open the file in a browser and try it.
3. This form also **has no action** on its `<form>` tag — what effect does that have when the "Save" button is pressed?

7. Summary & Further Exercises

7.1 Overall Summary of Jobsheet 1

File	Function	New Elements Learned
<code>index.html</code>	Home page + statistics summary	<code>header</code> , <code>nav</code> , <code>main</code> , <code>section</code> , <code>article</code> , <code>footer</code>
<code>books/list.html</code>	Book list table	<code>table</code> , <code>thead</code> , <code>tbody</code> , <code>tr</code> , <code>th</code> , <code>td</code> , <code>button type="button"</code>
<code>books/add.html</code>	Add book form	<code>form</code> , <code>label</code> , <code>input (text/number)</code> , <code>select/option</code> , <code>button type="submit"</code> , attributes <code>required/min/max</code>
<code>members/list.html</code>	Member list table (independent assignment)	Reapplying the table pattern to different data
<code>members/add.html</code>	Add member form (independent assignment)	Reapplying the form pattern to different data

In general, jobsheet 1 teaches **3 HTML patterns** that will be used again and again throughout the application:

1. **Page skeleton** (`header` + `nav` + `main` + `footer`) — the same on all 5 pages.
2. **Data table** (`table/thead/tbody/tr/th/td`) — used on both "list" pages.
3. **Input form** (`form/label/input/select/button`) — used on both "add" pages.

7.2 Remember: This Is Only the Skeleton

A few things are **deliberately not here** in this jobsheet:

- **No CSS** → the display is still plain, with no colour and no tidy layout.
- **No JavaScript** → the Edit and Delete buttons in the tables do not work yet.
- **The forms are not connected to anything** (`<form>` without `action`) → the "Save" button does not actually save data yet.
- **The table data is static (dummy) data**, typed manually into the HTML rather than pulled from a database.

All of this will start to be covered in the next jobsheet.

7.3 How to Try It Yourself

1. Open the `index.html` file directly in a browser (double-click it, or right-click → *Open with* → choose a browser). No server is needed yet.
2. Click the navigation menu in the header to move between pages. Notice how the `href` on each `<a>` determines the destination page.

3. Try filling in the form in `books/add.html` or `members/add.html`, then click "Save" — observe that nothing happens except a page reload (because there is no `action`).
4. Try emptying one of the fields that has the `required` attribute, then click "Save" — notice the browser showing HTML5's built-in automatic validation warning.

7.4 Exercises

To deepen your understanding, try the following:

1. **Complete the menu consistency** — add the "Member List" and "Add Member" links to the `<nav>` menu in `index.html`, `books/list.html`, and `books/add.html` (see the note in the `members/list.html` documentation, §5.4).
2. **Add 3 new rows of book data** in `books/list.html` by copying one `<tr>...</tr>` block and replacing its contents.
3. **Add a new column** to the member table, for example "Gender", complete with its `<th>` and `<td>` in every row.
4. **Add a new field** to the add member form, for example "Email", using `<input type="email">` (`type="email"` automatically validates the email address format, with no extra JavaScript needed).

Happy learning! If any part is still confusing, reread the core concepts in chapter 1, because almost all the technical terms used in the other chapters are explained there.